

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Лабораторная работа №1
Реализация REST API на основе boilerplate

Выполнил:
Христофоров Владислав

Группа
К3340

Поток
БР 1.2

Проверил:
Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Использовать шаблонный проект (boilerplate) для разработки серверного приложения. С учётом имеющихся данных по результатам домашнего задания №1 (ER-диаграмма) и домашнего задания №2 (спецификация API) реализовать модели данных, представления и контроллеры. В результате выполнения лабораторной работы продемонстрировать полностью работающее REST API приложение согласно заданному варианту.

Ход работы

1 Инфраструктура и настройка окружения

В качестве основы серверного приложения использован шаблонный проект на стеке TypeScript и фреймворка Express. База данных PostgreSQL развернута посредством Docker-контейнера (docker-compose.yml). Интегрирована библиотека multer для обработки загрузки медиафайлов (эндпоинт /upload). Для визуализации представлений и ручного тестирования эндпоинтов развернут интерфейс Swagger UI на базе файла openapi.yaml. В конфигурационном файле app.ts настроена автоматическая регистрация контроллеров и глобальная обработка ошибок.

2 Реализация моделей данных

С использованием ORM TypeORM спроектировано и реализовано 12 классов сущностей, образующих ядро системы:

- базовые и справочные сущности: User, Recipe, BlogPost, DishType, Ingredient,
- ассоциативные сущности: RecipeStep, RecipeIngredient, RecipeDishType, Comment, Like, SavedRecipe, Subscription.

В моделях реализованы следующие архитектурные решения:

- полиморфизм: для сущностей социальной активности (Like и Comment) настроены внешние ключи recipe_id и blog_post_id с параметром nullable: true, что позволило привязывать лайки и комментарии как к рецептам, так и к постам блога в рамках одних и тех же таблиц,

- строгая типизация в БД: для защиты от записи неконсистентных данных применен тип ENUM (например, единицы измерения ['g', 'ml', 'pcs', 'tbsp', 'tsp'] в RecipeIngredient и роли ['user', 'moderator', 'admin'] в User),
- мягкое удаление (Soft Delete): во всех контентных сущностях применен декоратор @DeleteDateColumn(), предотвращающий физическое удаление строк из базы данных (защита от потери информации).

3 Реализация представлений

Представления реализованы в виде Data Transfer Objects (DTO) с использованием библиотеки class-validator. При настройке слоя валидации и безопасности реализован следующий комплекс мер:

- разработаны классы валидации (например, CreateRecipeDto, RegisterDto), ограничивающие входящие данные с помощью декораторов @IsNotEmpty(), @IsString(), @Min(1) (для времени приготовления рецепта),
- в конфигурации сервера активирован параметр whitelist: true, который автоматически усекает любые несанкционированные поля из JSON-тел запросов, предотвращая уязвимость Mass Assignment,
- настроен процесс безопасного хранения учетных данных: реализован класс UserSubscriber, перехватывающий события beforeInsert и beforeUpdate для автоматического хэширования паролей криптографическим алгоритмом bcrypt (с проверкой наличия префикса \$2b\$, чтобы избежать двойного хэширования),
- настроена система безопасности на базе JWT-токенов: разработан authMiddleware, выполняющий валидацию токена из заголовка Authorization и внедряющий объект пользователя (id и role) в контекст HTTP-запроса.

4 Реализация контроллеров

На базе библиотеки routing-controllers разработано 8 контроллеров (AuthController, RecipeController, AdminController и др.), реализующих полный цикл CRUD-операций. В бизнес-логику маршрутизаторов внедрены следующие ключевые механизмы:

- транзакционность: эндпоинты создания и обновления рецептов (POST /recipes, PATCH /recipes/:id) реализованы с применением интерфейса queryRunner. Это обеспечило атомарное сохранение в базу данных самого рецепта, его пошаговых инструкций (RecipeStep), списка ингредиентов (RecipeIngredient) и категорий блюд (RecipeDishType) в рамках единой SQL-транзакции;

- продвинутая фильтрация: в методе getAllRecipes интегрирован QueryBuilder. Реализован полнотекстовый поиск (ILIKE) по названию и описанию. Для корректной фильтрации рецептов по массиву ингредиентов и категориям блюд применены SQL-подзапросы (subQuery), что исключило известную проблему усечения присоединенных таблиц при использовании оператора LEFT JOIN;

- ролевая модель (RBAC): разработан roleMiddleware. В контроллере AdminController реализованы защищенные методы модерации контента, назначения ролей (PATCH /users/:id/role) и управления блокировками (DELETE /users/:id/ban), доступные исключительно пользователям с ролями admin или moderator. Интегрирована логика безопасного восстановления аккаунтов через эндпоинт авторизации с проверкой флага is_banned.

Вывод

В ходе выполнения лабораторной работы успешно разработано и протестировано полнофункциональное серверное REST API приложение на основе выбранного шаблона (boilerplate). В полном объеме спроектированы и интегрированы модели базы данных, представления для обмена информацией и контроллеры для обработки бизнес-логики. Настроены механизмы строгой валидации, безопасной аутентификации и ролевого разграничения доступа. Разработанный программный код полностью реализует заявленную бизнес-логику и удовлетворяет требованиям целевого варианта.